

Assignment 4: Becoming an Independent Data Scientist

1. Region and Domain

State the region and the domain category that your data sets are about.

The **NYC Yellow Taxi Dataset** is a public dataset that provides detailed records of taxi trips in New York City. Here is a breakdown of the region and domain category:

Region:

The dataset covers taxi trips specifically within **New York City (NYC)**, including all five boroughs—Manhattan, Brooklyn, Queens, The Bronx, and Staten Island. Some trips may extend slightly outside of NYC for destinations such as nearby airports or surrounding areas.

Domain Category: The domain of the dataset falls under transportation, specifically in the area of urban mobility and transportation analytics. It is often used for studying traffic patterns, transportation demand, ride-hailing services, public policy, and other areas of interest like urban planning, economics, and sustainability.

The data includes information such as pickup and dropoff locations, times, fare amounts, and passenger counts, among other features.

2. Research Question

You must state a question about the domain category and region that you identified as being interesting.

How have the **number of passengers (passenger density)** in the **yellow taxis** for **NYC** for **different TLC Taxi Zones** (e.g., JFK Airport, Midtown Center, Penn Station/Madison Sq West etc.) in the month **April** of the year **2023** differ over with respect to different

1. **zones**
2. **times (hours)** in a day
3. **pickup** vs. **dropoff**
4. **weekdays** vs. **weekends**?

3. Links

You must provide at least two links to publicly accessible datasets. These could be links to files such as CSV or Excel files, or links to websites which might have data in tabular form, such as Wikipedia pages.

Here are couple of publicly accessible sources where we can find the **NYC Yellow Taxi Dataset** (for the year **2023** and month **April**):

1. NYC OpenData

https://data.cityofnewyork.us/Transportation/2023-Yellow-Taxi-Trip-Data/4b4i-vvec/data_preview

2. TLC Trip Record Data

- <https://www.nyc.gov/site/tlc/about/data-and-research.page>
- https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

4. Image

You must upload an image which addresses the research question you stated. In addition to addressing the question, this visual should follow Cairo's principles of truthfulness, functionality, beauty, and insightfulness.

5. Discussion

You must contribute a short (1-2 paragraph) written justification of how your visualization addresses your stated research question.

This visualization was concerned with answering the question of how the win percentages of the four major Detroit sports teams (The Detroit Pistons, The Red Wings, The Detroit Tigers, and The Detroit Lions) have changed over the last 40 years. Wikipedia was scraped for data concerning wins and losses by season for each team. Any tie games were dropped from the analysis in order to provide a more level comparison between the sports. A 10 year moving average was plotted to help the reader identify any major trends in the team's win percentages. The plot indicates years of high win percentage for The Detroit Pistons around 1990 and 2005, with periods of decline following both high points. The Detroit Red Wings' win percentage shows an increase in the 1990's and stays high through the early 2000's, with a downward trend in recent years. The Detroit Tiger's win percentage shows less variation than the other teams, hovering around 50%, with it's lowest win percentage in the early 2000's. The win percentage of the Detroit Lions shows low levels throughout the early 2000's, with an increase after 2010.

```
In [1]: # https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
import pandas as pd
df_traffic = pd.read_parquet('data/yellow_tripdata_2023-04.parquet', engine='fastparquet')
df_traffic.to_csv('data/yellow_tripdata_2023-04.csv')
df_traffic.head()
```

Out[1]:

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance
0	1.0	2023-04-01 00:14:49	2023-04-01 00:45:01	2.0	4
1	2.0	2023-04-01 00:00:24	2023-04-01 00:56:19	1.0	21
2	1.0	2023-04-01 00:03:50	2023-04-01 00:14:42	2.0	1
3	1.0	2023-04-01 00:53:18	2023-04-01 01:01:28	1.0	1
4	2.0	2023-04-01 00:07:00	2023-04-01 00:17:16	2.0	1

In [2]:

```
df_PU = df_traffic.groupby('PULocationID').agg({'passenger_count': 'sum'}).reset_index()
df_DO = df_traffic.groupby('DOLocationID').agg({'passenger_count': 'sum'}).reset_index()
df_PU.head()
```

Out[2]:

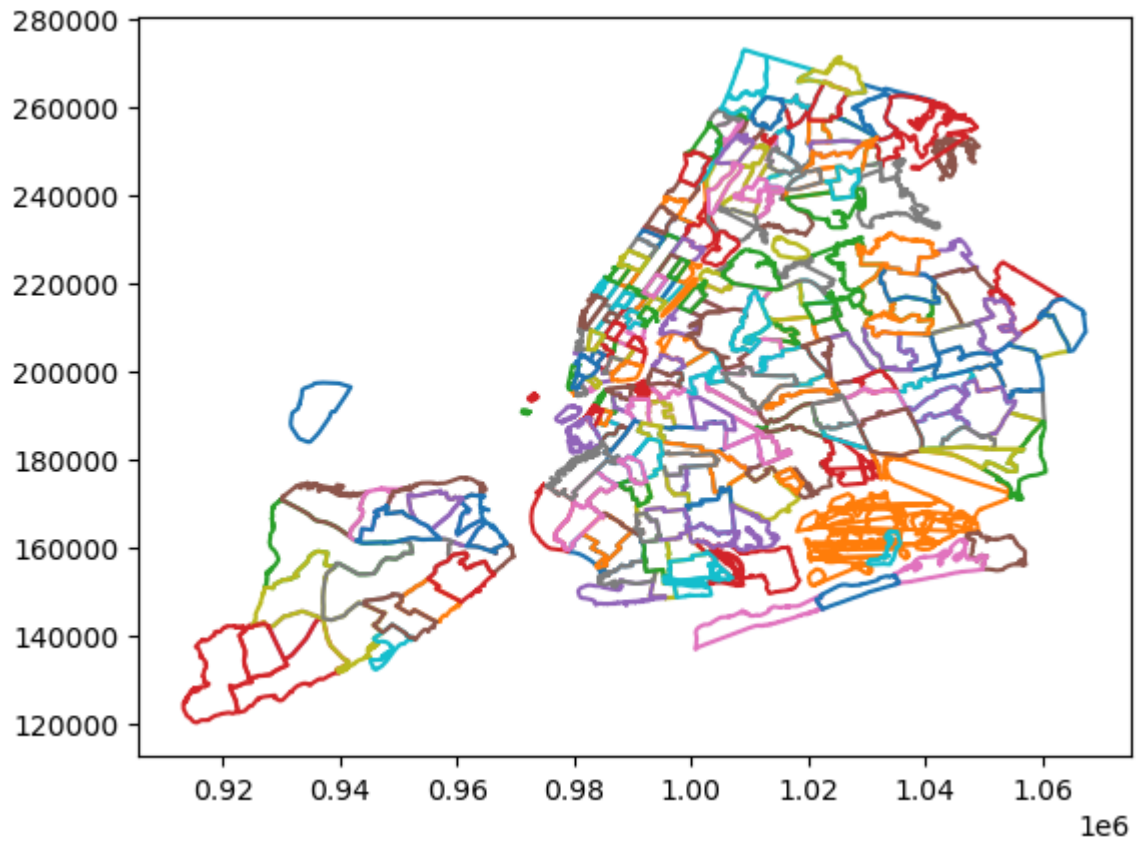
	PULocationID	passenger_count
0	1.0	798.0
1	2.0	2.0
2	3.0	61.0
3	4.0	4769.0
4	5.0	41.0

In [14]:

```
import shapefile as shp # Requires the pyshp package
import matplotlib.pyplot as plt

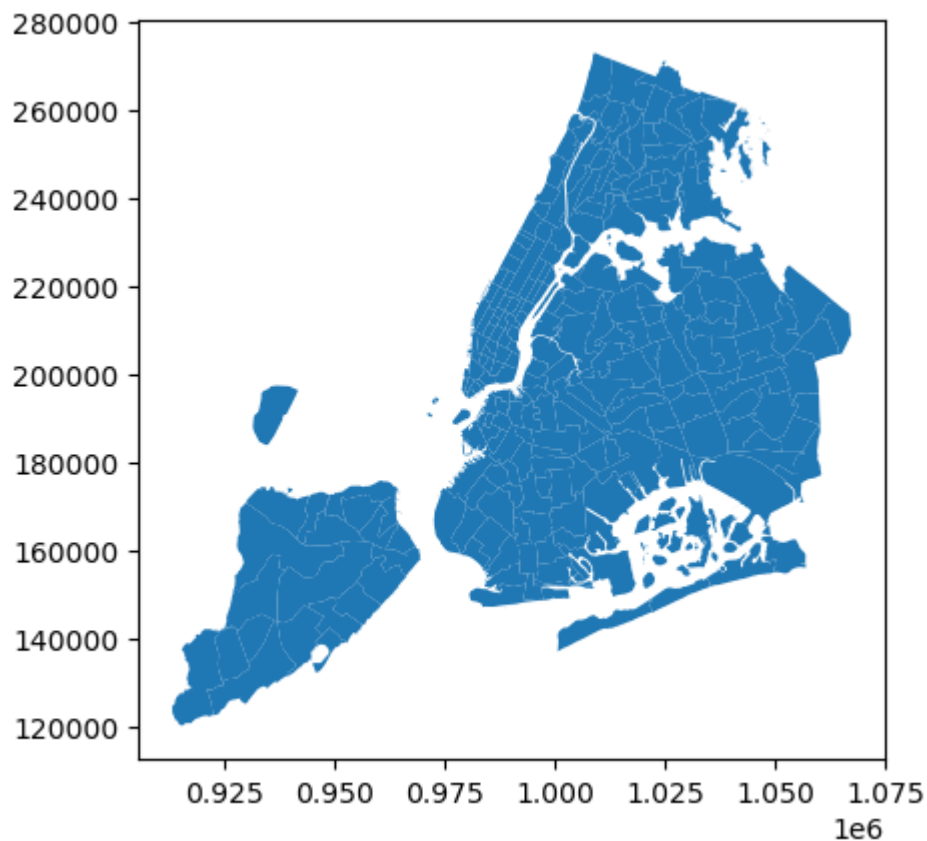
sf = shp.Reader("taxi_zones/taxi_zones.shp")

plt.figure()
for shape in sf.shapeRecords():
    x = [i[0] for i in shape.shape.points[:]]
    y = [i[1] for i in shape.shape.points[:]]
    plt.plot(x,y)
plt.show()
```



```
In [3]: import geopandas as gpd
df_map = gpd.read_file("taxi_zones/taxi_zones.shp")
df_map.plot()
```

Out[3]: <Axes: >



```
In [4]: df_map.head()
```


Out[4]:

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	ge
0	1	0.116357	0.000782	Newark Airport	1	EWR	PC ((933 192 933091.
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPC (((1033 172 1
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	PC ((102 256 1026495
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	PC ((992 203 992068.
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	PC ((93 144 936



In [19]: `len(df.LocationID.unique())`

Out[19]: 260

In [20]: `df_loc = pd.read_csv('data/taxi_zone_lookup.csv')`
`df_loc.head()`

Out[20]:

	LocationID	Borough	Zone	service_zone
0	1	EWR	Newark Airport	EWR
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

In [37]: `df_map.crs`

```
Out[37]: <Projected CRS: EPSG:2263>
Name: NAD83 / New York Long Island (ftUS)
Axis Info [cartesian]:
- X[east]: Easting (US survey foot)
- Y[north]: Northing (US survey foot)
Area of Use:
- name: United States (USA) - New York - counties of Bronx; Kings; Nassau; New
York; Queens; Richmond; Suffolk.
- bounds: (-74.26, 40.47, -71.8, 41.3)
Coordinate Operation:
- name: SPCS83 New York Long Island zone (US Survey feet)
- method: Lambert Conic Conformal (2SP)
Datum: North American Datum 1983
- Ellipsoid: GRS 1980
- Prime Meridian: Greenwich
```

```
In [53]: df_wm = df_map.to_crs(epsg=3857)
```

```
In [61]: df_map_PU = df_map.merge(df_PU, left_on='LocationID', right_on='PULocationID')
df_map_PU.sort_values('passenger_count', ascending=False).head(10)
```

Out[61]:

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough		
	126	132	0.245479	0.002038	JFK Airport	132	Queens	MULTIPOINT (((108 40.641111, -73.778056, 1 1
	155	161	0.035804	0.000072	Midtown Center	161	Manhattan	MULTIPOINT (((108 40.758056, -73.98909 2
	229	237	0.042213	0.000096	Upper East Side South	237	Manhattan	MULTIPOINT (((108 40.758056, -73.98935 2
	228	236	0.044252	0.000103	Upper East Side North	236	Manhattan	MULTIPOINT (((108 40.758056, -73.98935 2
	222	230	0.031028	0.000056	Times Sq/Theatre District	230	Manhattan	MULTIPOINT (((108 40.758056, -73.9886 2
	136	142	0.038176	0.000076	Lincoln Square East	142	Manhattan	MULTIPOINT (((108 40.758056, -73.9893 2
	132	138	0.107467	0.000537	LaGuardia Airport	138	Queens	MULTIPOINT (((108 40.641111, -73.778056, 2
	156	162	0.035270	0.000048	Midtown East	162	Manhattan	MULTIPOINT (((108 40.758056, -73.9920 2
	178	186	0.024696	0.000037	Penn Station/Madison Sq West	186	Manhattan	MULTIPOINT (((108 40.758056, -73.9866 2
	157	163	0.034177	0.000041	Midtown North	163	Manhattan	MULTIPOINT (((108 40.758056, -73.9900 2

```
In [29]: import matplotlib
import matplotlib.pyplot as plt
import contextily as cx

def plot_density(df, type):
    #pd.options.display.mpl_style = 'default' #Better Styling
    df_wm = df.to_crs(epsg=3857)
```

```

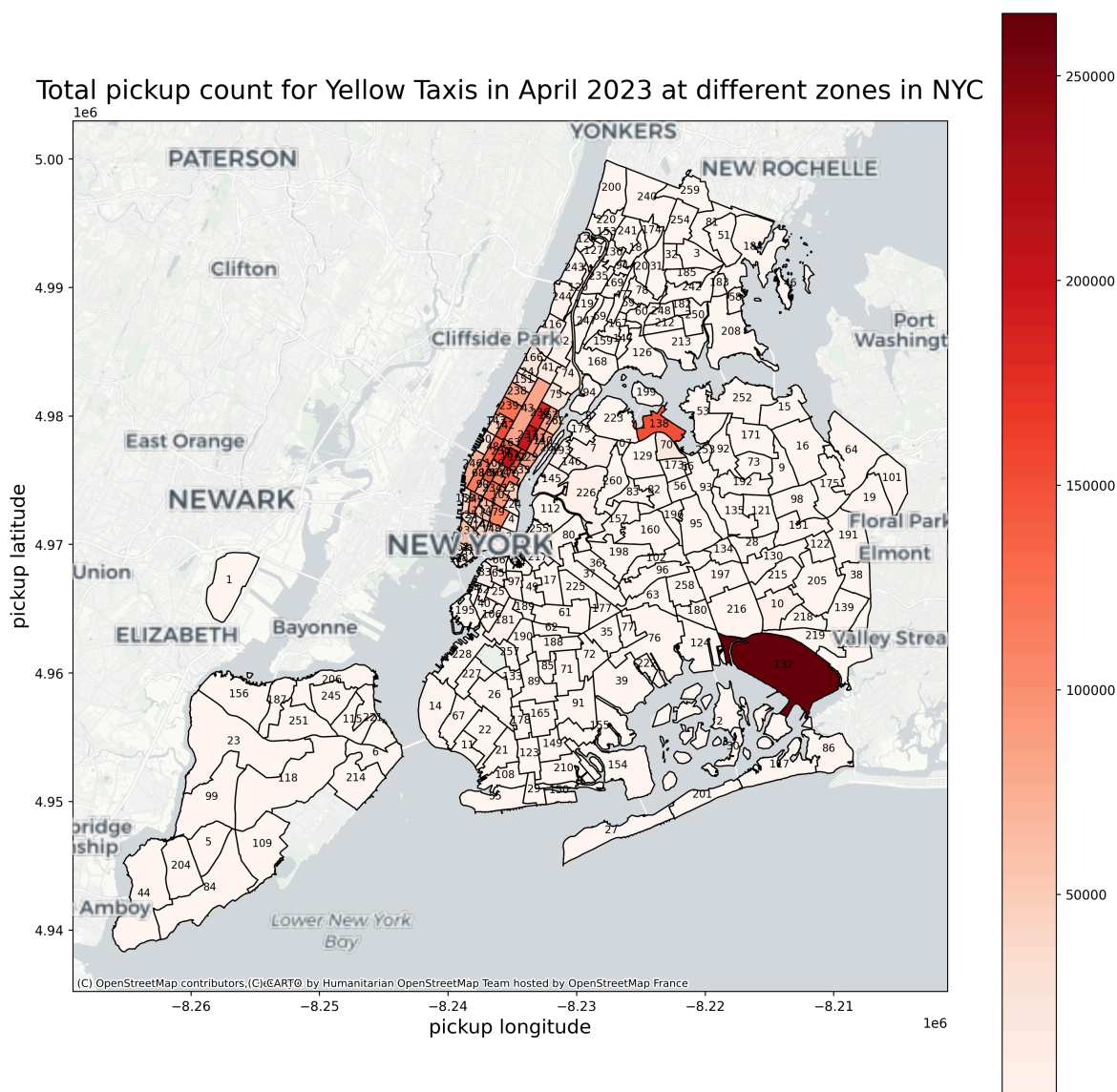
new_style = {'grid': False} #Remove grid
matplotlib.rcParams['axes', **new_style)
plt.rcParams['figure.figsize'] = (17.5, 17) #Size of figure
plt.rcParams['figure.dpi'] = 250
#df['passenger_count'] = df['passenger_count'] / df['passenger_count'].max()
ax = df_wm.plot(figsize=(15, 15), column='passenger_count', cmap='Reds', edge
df_wm.apply(lambda x: ax.annotate(text=x['LocationID'], xy=x.geometry.centro
cx.add_basemap(ax, crs=df_map_PU.crs)
cx.add_basemap(ax, source=cx.providers.CartoDB.PositronNoLabels, zoom=12)
cx.add_basemap(ax, source=cx.providers.CartoDB.PositronOnlyLabels, zoom=10)
plt.xlabel('{} longitude'.format(type), size=15)
plt.ylabel('{} latitude'.format(type), size=15)
plt.title('Total {} count for Yellow Taxis in April 2023 at different zones
plt.show()

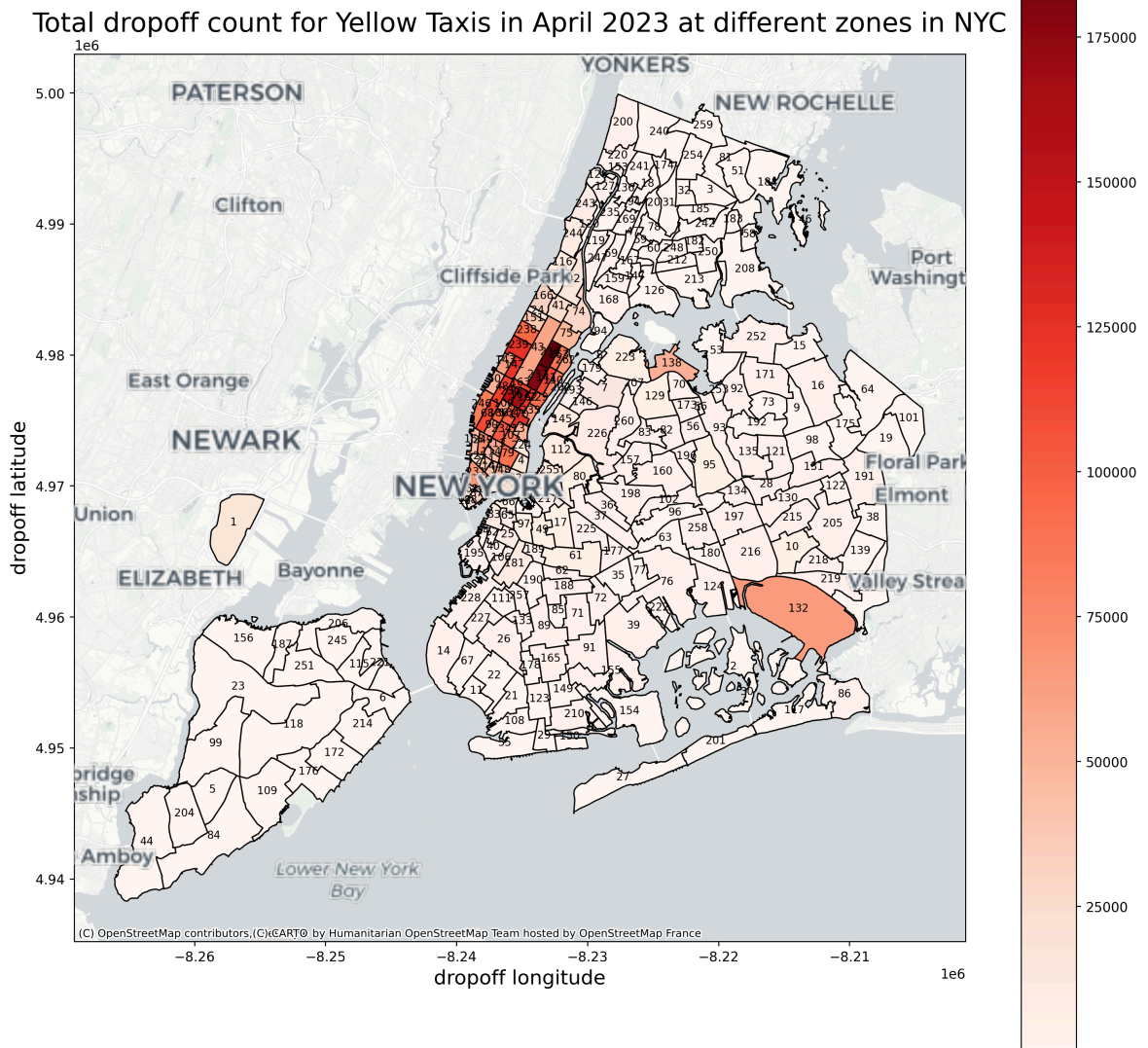
```

```

In [5]: df_map_PU = df_map.merge(df_PU, left_on='LocationID', right_on='PULocationID')
plot_density(df_map_PU, 'pickup')
df_map_DO = df_map.merge(df_DO, left_on='LocationID', right_on='DOLocationID')
plot_density(df_map_DO, 'dropoff')

```





```
In [10]: df_traffic['PIweekend'] = df_traffic.tpep_pickup_datetime.dt.dayofweek > 4 #df_t
df_PU_weekend = df_traffic[df_traffic['PIweekend'] == True]
df_PU_weekend_loc = df_PU_weekend.groupby('PULocationID').agg({'passenger_count'
df_PU_weekday = df_traffic[df_traffic['PIweekend'] == False]
df_PU_weekday_loc = df_PU_weekday.groupby('PULocationID').agg({'passenger_count'
df_PU_weekend_loc.sort_values('passenger_count', ascending=False).head(5)
```

```
Out[10]:
```

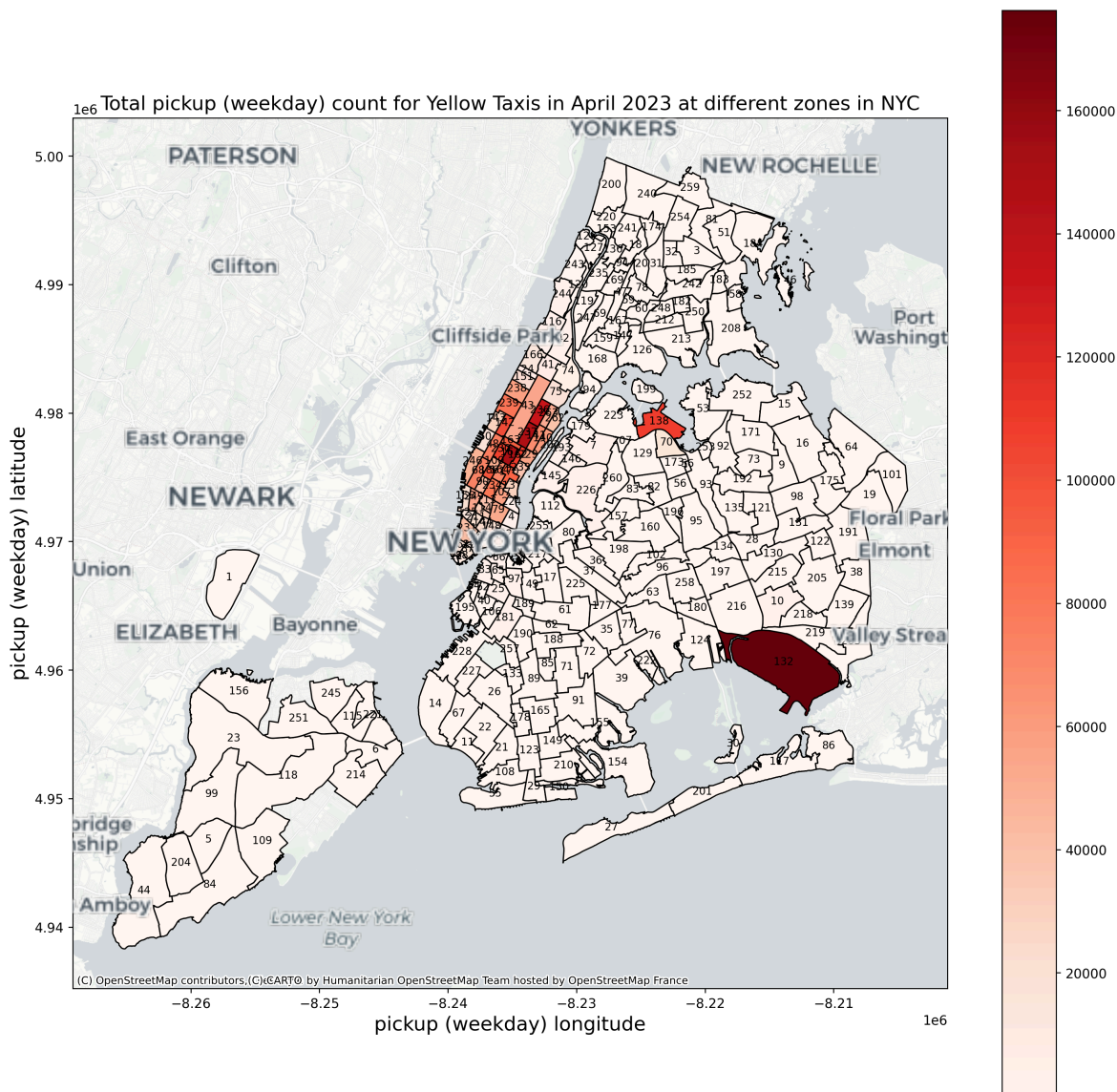
	PULocationID	passenger_count
123	132.0	89051.0
215	230.0	56490.0
76	79.0	55667.0
152	161.0	53448.0
222	237.0	53086.0

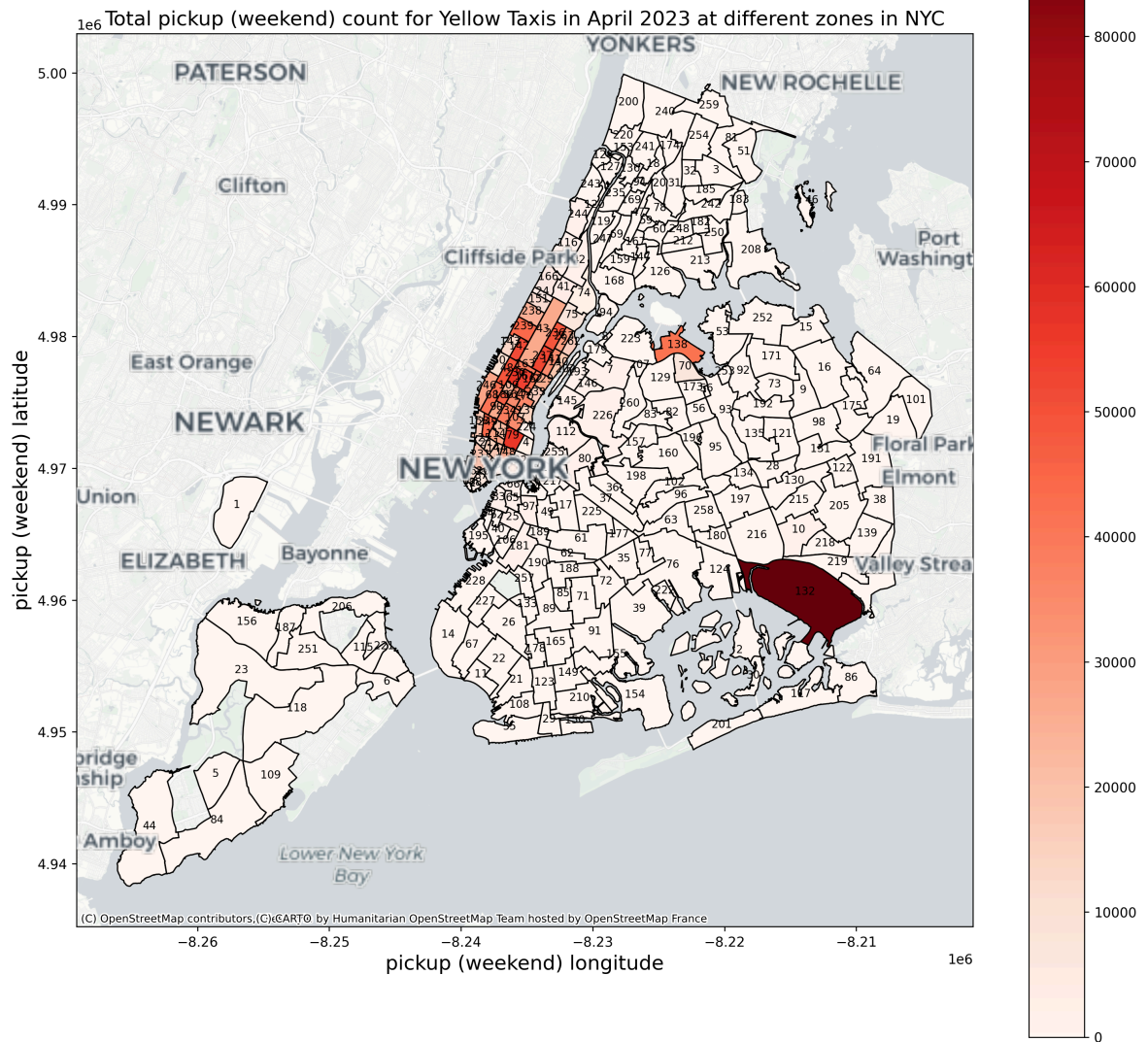
```
In [8]: df_PU_weekday_loc.sort_values('passenger_count', ascending=False).head(5)
```


Out[8]:

	PULocationID	passenger_count
125	132.0	176271.0
154	161.0	150004.0
226	237.0	148193.0
225	236.0	131997.0
131	138.0	109076.0

```
In [30]: df_map_PU = df_map.merge(df_PU_weekday_loc, left_on='LocationID', right_on='PULocationID')
df_map_PU.plot_density(df_map_PU, 'pickup (weekday)')
df_map_PU = df_map.merge(df_PU_weekend_loc, left_on='LocationID', right_on='PULocationID')
df_map_PU.plot_density(df_map_PU, 'pickup (weekend)')
```





```
In [16]: df_PU_weekend['hour'] = df_PU_weekend.tpep_pickup_datetime.dt.hour
df_PU_weekend_hr = df_PU_weekend.groupby('hour').agg({'passenger_count': 'sum'})
df_PU_weekend_hr['type'] = 'weekend'
df_PU_weekend_hr
```

C:\Users\hp\AppData\Local\Temp\ipykernel_9384\4009435247.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_PU_weekend['hour'] = df_PU_weekend.tpep_pickup_datetime.dt.hour
```

Out[16]:

	hour	passenger_count	type
0	0	75048.0	weekend
1	1	59734.0	weekend
2	2	43915.0	weekend
3	3	27743.0	weekend
4	4	13707.0	weekend
5	5	5995.0	weekend
6	6	10801.0	weekend
7	7	17194.0	weekend
8	8	28050.0	weekend
9	9	45686.0	weekend
10	10	62741.0	weekend
11	11	77350.0	weekend
12	12	86122.0	weekend
13	13	87653.0	weekend
14	14	87691.0	weekend
15	15	88820.0	weekend
16	16	94185.0	weekend
17	17	97047.0	weekend
18	18	94165.0	weekend
19	19	85642.0	weekend
20	20	77247.0	weekend
21	21	72349.0	weekend
22	22	71298.0	weekend
23	23	63980.0	weekend

In [17]:

```
df_PU_weekday['hour'] = df_PU_weekday.tpep_pickup_datetime.dt.hour
df_PU_weekday_hr = df_PU_weekday.groupby('hour').agg({'passenger_count': 'sum'})
df_PU_weekday_hr['type'] = 'weekday'
df_PU_weekday_hr
```

C:\Users\hp\AppData\Local\Temp\ipykernel_9384\3149654141.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

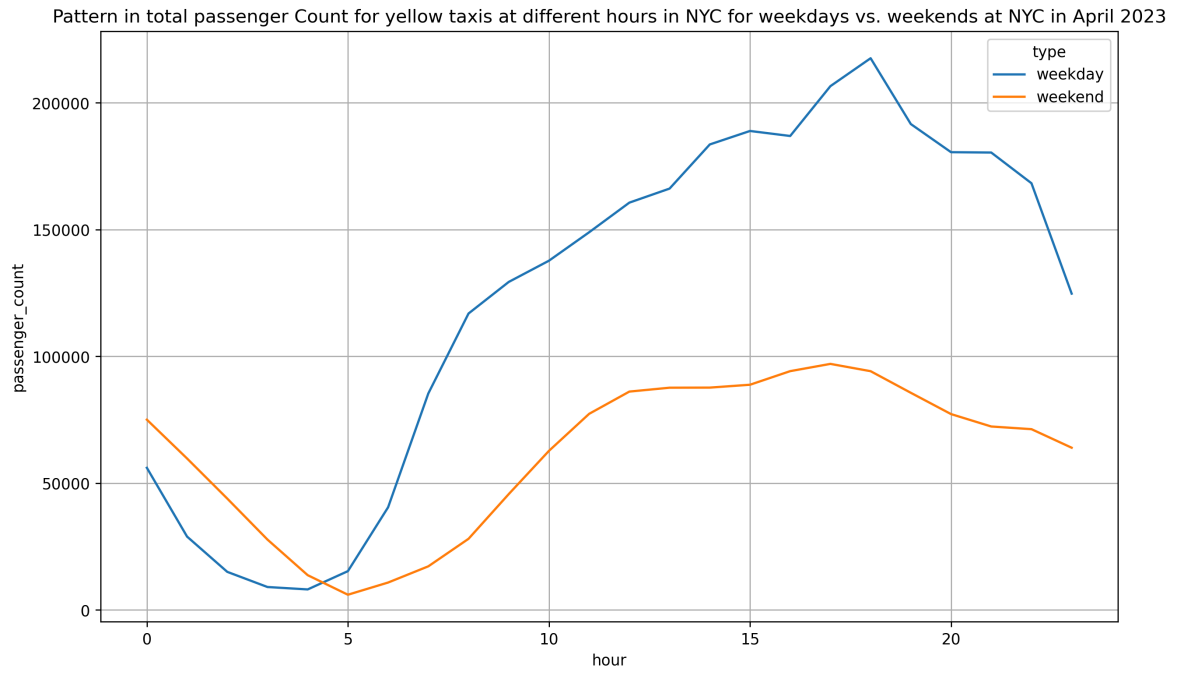
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_PU_weekday['hour'] = df_PU_weekday.tpep_pickup_datetime.dt.hour
```


Out[17]:

	hour	passenger_count	type
0	0	56059.0	weekday
1	1	28926.0	weekday
2	2	15012.0	weekday
3	3	9035.0	weekday
4	4	8084.0	weekday
5	5	15294.0	weekday
6	6	40489.0	weekday
7	7	85382.0	weekday
8	8	116917.0	weekday
9	9	129372.0	weekday
10	10	137745.0	weekday
11	11	148968.0	weekday
12	12	160656.0	weekday
13	13	166194.0	weekday
14	14	183603.0	weekday
15	15	188921.0	weekday
16	16	186943.0	weekday
17	17	206602.0	weekday
18	18	217610.0	weekday
19	19	191647.0	weekday
20	20	180553.0	weekday
21	21	180413.0	weekday
22	22	168306.0	weekday
23	23	124756.0	weekday

```
In [28]: import seaborn as sns
df_PU_week = pd.concat([df_PU_weekday_hr, df_PU_weekend_hr])
plt.figure(figsize=(12,7))
sns.lineplot(data=df_PU_week, x="hour", y="passenger_count", hue="type")
plt.title('Pattern in total passenger Count for yellow taxis at different hours')
plt.grid()
plt.show()
```



```
In [25]: df_PU_week.head()
```

Out[25]:

	hour	passenger_count	type
0	0	56059.0	weekday
1	1	28926.0	weekday
2	2	15012.0	weekday
3	3	9035.0	weekday
4	4	8084.0	weekday